

The output is:

```
[1, 2, 3, 4]
```

In the collection, the list data type is retrieved using the 'pop' method, and data can be inserted using the 'insert' method:

```
#Pop
userL.pop()
print(userL.data)
```

```
#Insert
```

```
userL.insert(0,5)
print(userL.data)
```

```
[1, 2, 3] # Pop from the left
[5, 1, 2, 3] #insert at 0th position
```

Collections: String module

Like the list, the collections package also supports string data types. This package provides various string methods, like list, as mentioned earlier. A few of these methods are shown here to give an idea of the functionalities of the collections package. The use of string data requires the *UserString* module of the package.

```
from collections import UserString
```

As mentioned in the list, all the methods of the module *UserString* can be used as shown below:

```
l = "open source for you 2023"
users = UserString(l)
print(type(users))
print(users.data)
```

The output is:

```
<class 'collections.UserString'>
open source for you 2023
```

Some useful methods of *UserString* are 'count()', 'find()' and 'index()', where 'count' is used to count the number of occurrences of a string. The 'find()' and 'index()' are used to locate a string position. The *capitalize()* function is a handy function for capitalising the first character of a given string. Functions like 'isalpha()', 'isascii()', and 'isalnum()' are boolean functions to verify an alphabetic, ASCII, and alphanumeric string, respectively. All this can be done as follows.

Count the number of occurrences of 'o' within the given string:

```
print(users.count("o"))
```

Similarly, to find the index position of a string 'r', one can use 'find()' or 'index()':

```
print(users.find("r"))
```

```
print(users.index("r"))
```

The outputs are:

```
8
8
```

To capitalise the first character of a string, the *capitalize()* method can be used as follows:

```
print(users.capitalize())
```

The output is:

```
Open source for you 2023
```

Boolean functions 'isalpha()', 'isascii()', and 'isalnum()' can be used as shown below:

```
print(users.isalpha())
```

```
print(users.isascii())
```

```
print(users.isalnum())
```

Collections: Default dictionary

The default dictionary of the 'collections' module is an added advantage over the ordinary dictionary structure. Where an ordinary dictionary variable gives an error message for a non-existing key-value pair, the 'defaultdict' module of collections produces a valid customised response for a non-existing key value.

In the ordinary dictionary the outputs of:

```
d = { "a":1,"b":1}
print(d["a"])
print(d["c"])
```

...the output are:

```
1
Traceback (most recent call last):
  File "D:\SENTIMENT_ANALYSIS\tp_2.py", line 35, in <module>
    print(d["c"])
  KeyError: 'c'
```